

Dongyeon Kim (Leo)

2026.09.13

CSSE 313 Artificial Intelligence

Lab #1 Reflection

- **Predictions for A–B, committed before running:**

Experiment:

1. AND models (STEP) learning rate = 0.01

What I will change:

1. I will change the learning rate of the perceptron to 0.01.
2. All else held fixed to:
 - a. Trained till error threshold; max epoch = 500; error threshold = 0.05
 - b. Initial weight = [0.0, 0.0] (with 2 inputs)
 - c. Initial bias = 0.0
 - d. STEP threshold = 0.0
 - e. Gradient decent = false

What I predict will happen:

1. Smaller learning rate on the model would require more epochs to reach the error threshold computed by “target - prediction”.

Why (mechanism I expect):

1. Learning rate would control how the weights would change on each computation of new weights for the next epoch. With the learning rate being so small, every weight should be updated on a smaller magnitude. Smaller updates would mean that the weights will approach the correct target slower, so the STEP perceptron model would need

proportionally more epochs to reach the error threshold. However, because 500 epochs is a generous number, the model should still comfortably converge before hitting the cap. I'd expect STEP to converge somewhat earlier than sigmoid, since STEP's update is a clean $+1/0/-1$ error signal while sigmoid's error signal gets squashed by the derivative term.

What would surprise me:

1. If learning rate and epochs did not have proportional relationship.

Experiment:

1. AND models (Sigmoid) learning rate = 0.01

What I will change:

1. I will change the learning rate of the perceptron to 0.01.
2. All else held fixed to:
 - a. Trained till error threshold; max epoch = 500; error threshold = 0.05
 - b. Initial weight = [0.0, 0.0] (with 2 inputs)
 - c. Initial bias = 0.0
 - d. Gradient decent = true

What I predict will happen:

1. Smaller learning rate on the model would require more epochs to reach the error threshold computed by "target - prediction".

Why (mechanism I expect):

1. Learning rate would control how the weights would change on each computation of new weights for the next epoch. With the learning rate being so small, every weight should be

updated on a smaller magnitude. Smaller updates would mean that the weights will approach the correct target slower, so the Sigmoid perceptron model would need proportionally more epochs to reach the error threshold. However, because 500 epochs is a generous number, the model should still comfortably converge before hitting the cap.

What would surprise me:

1. If learning rate and epochs did not have proportional relationship.

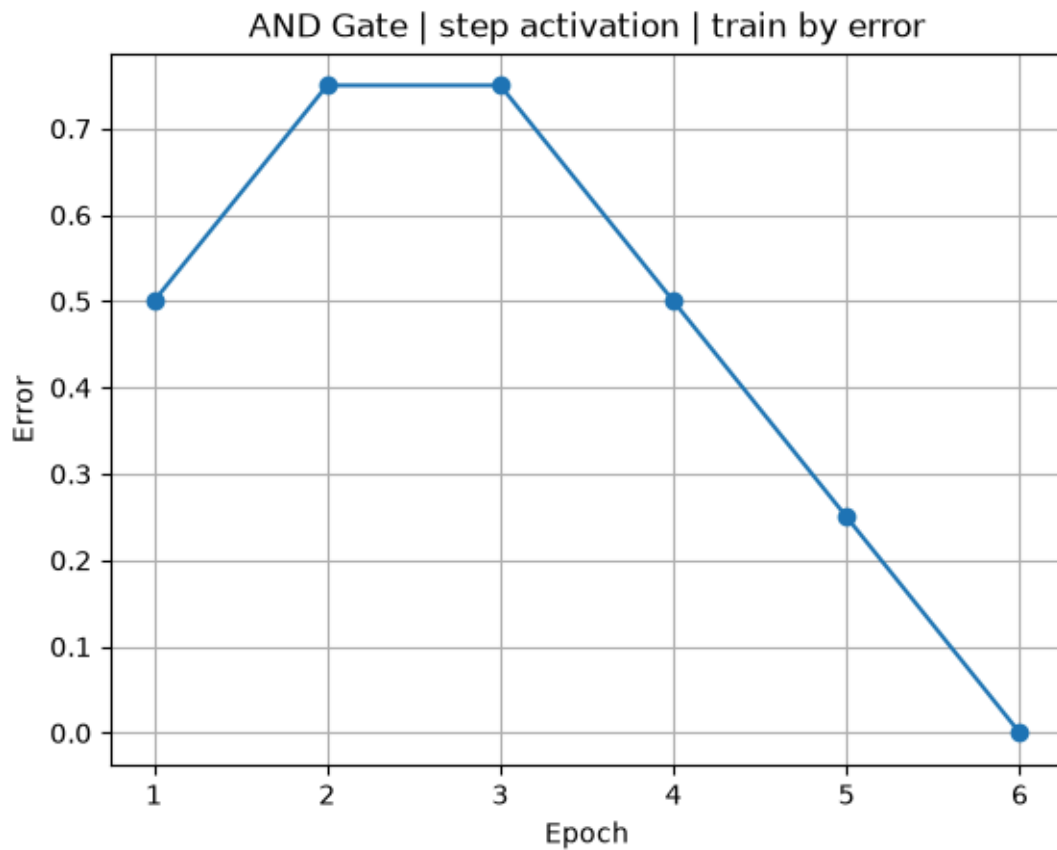
- **Experiment**

A:

1. Learning rate = 1

```
Epoch: 1/500 | Average Error: 0.5 | Weights: [1.0, 1.0]
Epoch: 2/500 | Average Error: 0.75 | Weights: [2.0, 1.0]
Epoch: 3/500 | Average Error: 0.75 | Weights: [2.0, 1.0]
Epoch: 4/500 | Average Error: 0.5 | Weights: [2.0, 2.0]
Epoch: 5/500 | Average Error: 0.25 | Weights: [2.0, 1.0]
Epoch: 6/500 | Average Error: 0.0 | Weights: [2.0, 1.0]
Training completed.

Final weights: [2.0, 1.0]
Final bias: -3.0
Inputs: [0, 0] | Target: 0 | Prediction: 0
Inputs: [0, 1] | Target: 0 | Prediction: 0
Inputs: [1, 0] | Target: 0 | Prediction: 0
Inputs: [1, 1] | Target: 1 | Prediction: 1
```



2. Learning rate = 0.5

```
Epoch: 1/500 | Average Error: 0.5 | Weights: [0.5, 0.5]
Epoch: 2/500 | Average Error: 0.75 | Weights: [1.0, 0.5]
Epoch: 3/500 | Average Error: 0.75 | Weights: [1.0, 0.5]
Epoch: 4/500 | Average Error: 0.5 | Weights: [1.0, 1.0]
Epoch: 5/500 | Average Error: 0.25 | Weights: [1.0, 0.5]
Epoch: 6/500 | Average Error: 0.0 | Weights: [1.0, 0.5]
Training completed.
```

```
Final weights: [1.0, 0.5]
```

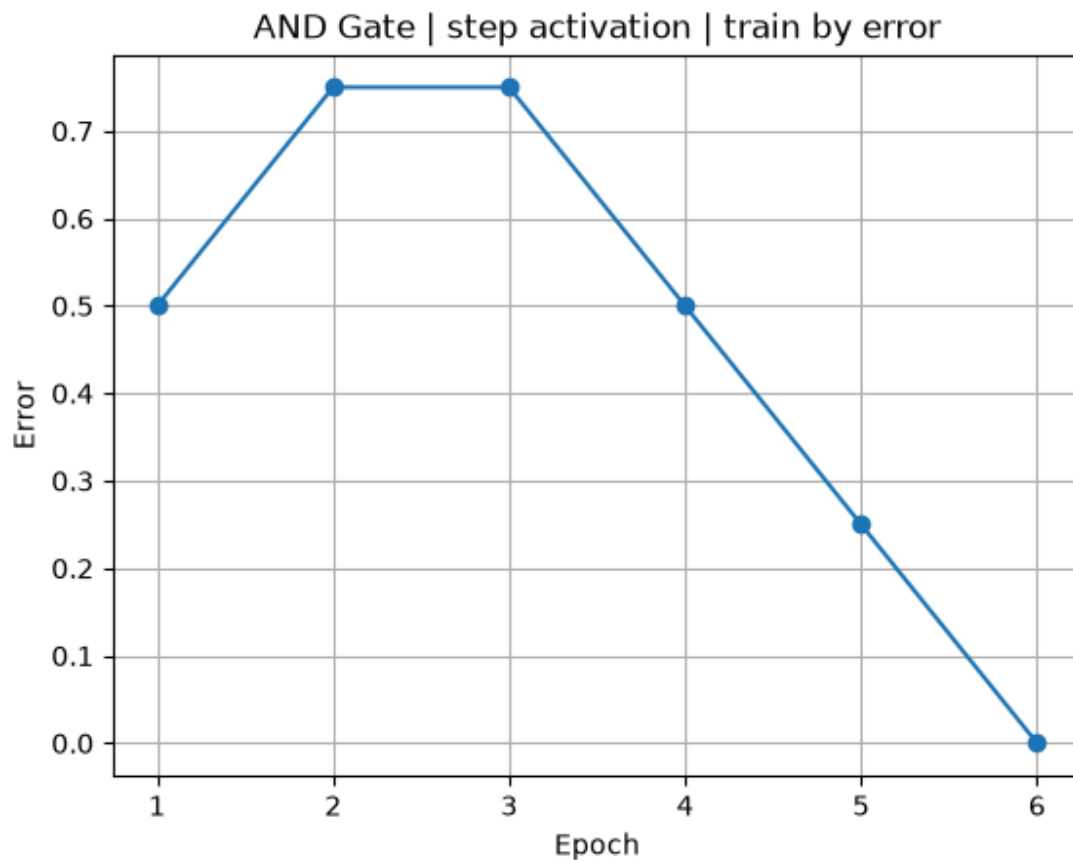
```
Final bias: -1.5
```

```
Inputs: [0, 0] | Target: 0 | Prediction: 0
```

```
Inputs: [0, 1] | Target: 0 | Prediction: 0
```

```
Inputs: [1, 0] | Target: 0 | Prediction: 0
```

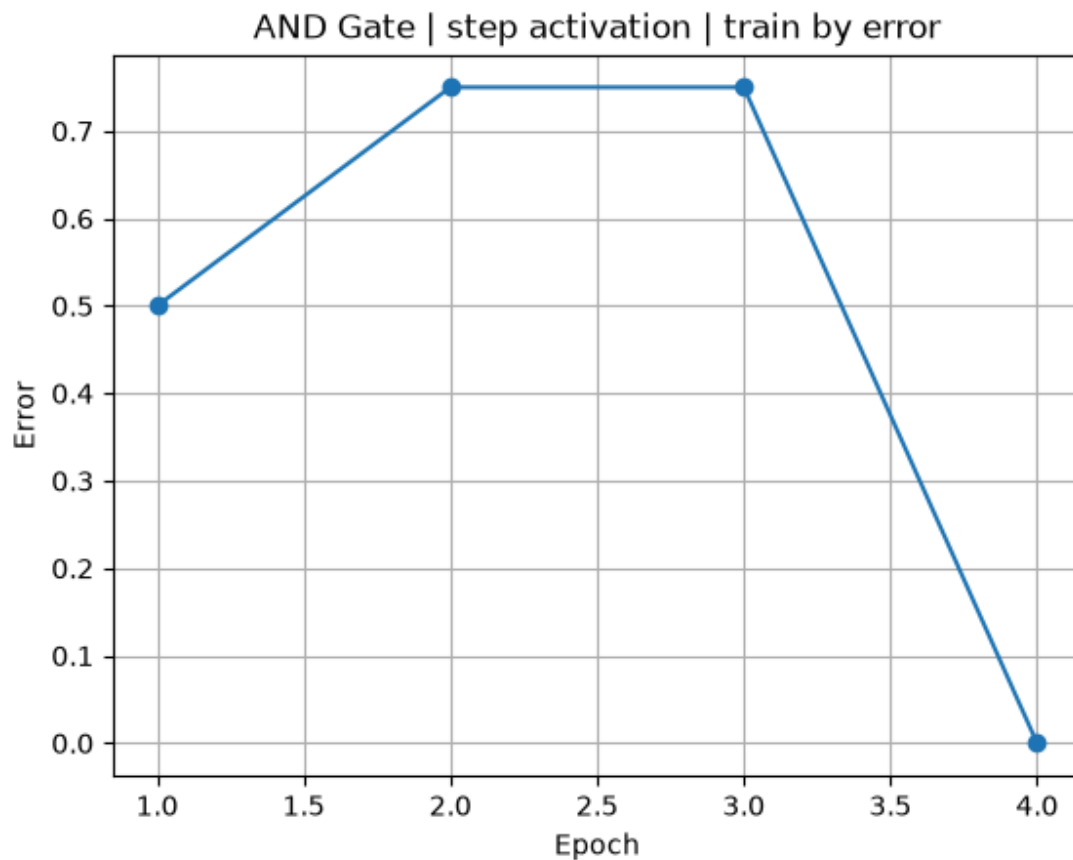
```
Inputs: [1, 1] | Target: 1 | Prediction: 1
```



3. Learning rate = 0.1

```
Epoch: 1/500 | Average Error: 0.5 | Weights: [0.1, 0.1]
Epoch: 2/500 | Average Error: 0.75 | Weights: [0.2, 0.1]
Epoch: 3/500 | Average Error: 0.75 | Weights: [0.2, 0.1]
Epoch: 4/500 | Average Error: 0.0 | Weights: [0.2, 0.1]
Training completed.
```

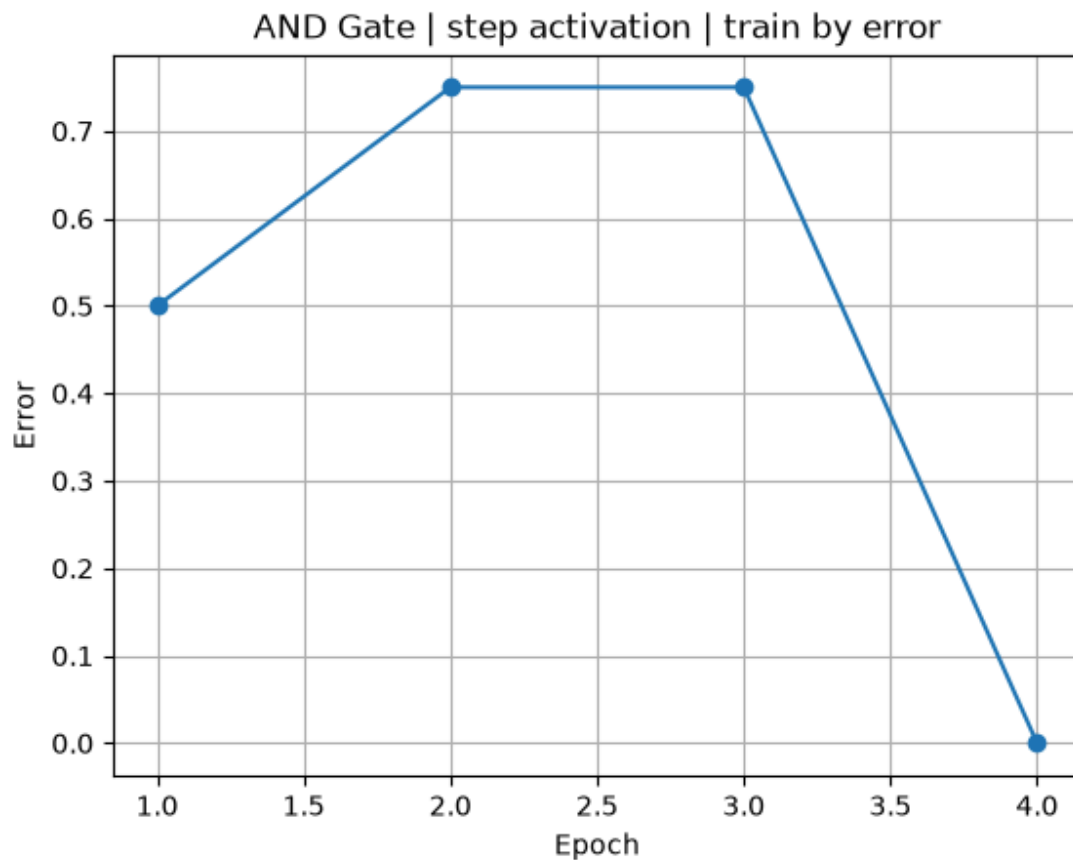
```
Final weights: [0.2, 0.1]
Final bias: -0.20000000000000004
Inputs: [0, 0] | Target: 0 | Prediction: 0
Inputs: [0, 1] | Target: 0 | Prediction: 0
Inputs: [1, 0] | Target: 0 | Prediction: 0
Inputs: [1, 1] | Target: 1 | Prediction: 1
```



4. Learning rate = 0.05

```
Epoch: 1/500 | Average Error: 0.5 | Weights: [0.05, 0.05]
Epoch: 2/500 | Average Error: 0.75 | Weights: [0.1, 0.05]
Epoch: 3/500 | Average Error: 0.75 | Weights: [0.1, 0.05]
Epoch: 4/500 | Average Error: 0.0 | Weights: [0.1, 0.05]
Training completed.
```

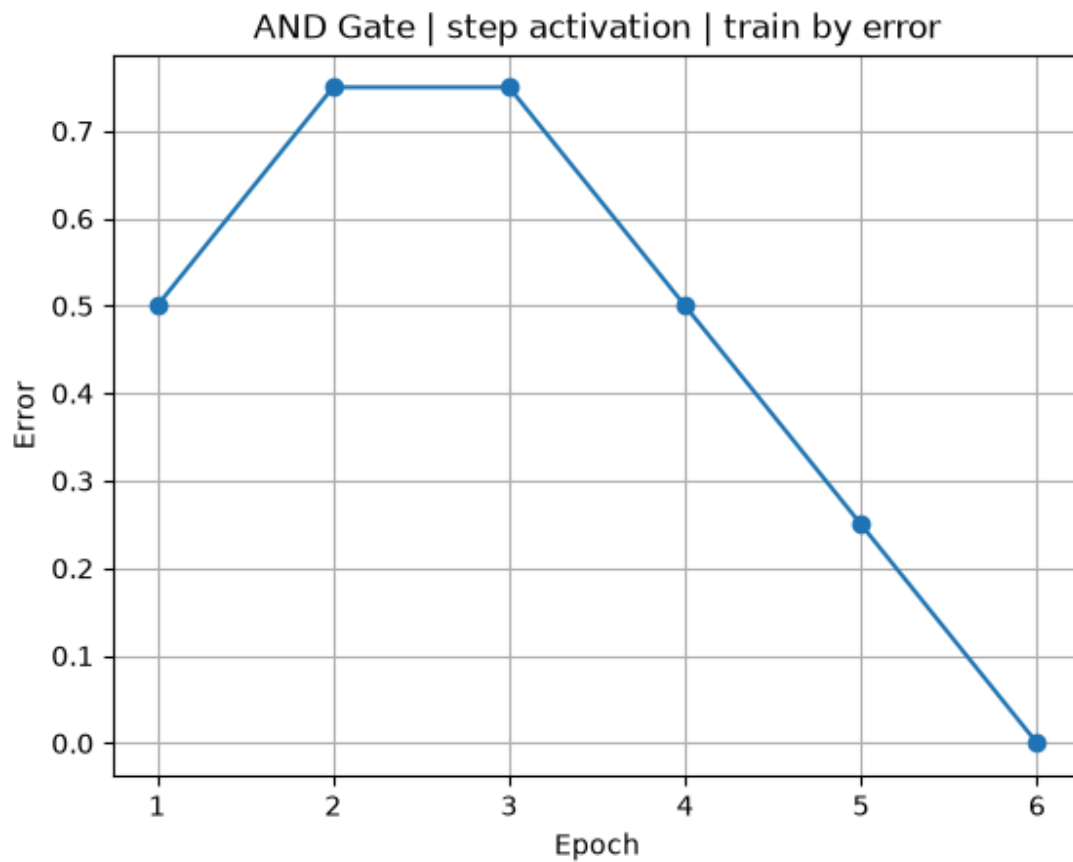
```
Final weights: [0.1, 0.05]
Final bias: -0.10000000000000002
Inputs: [0, 0] | Target: 0 | Prediction: 0
Inputs: [0, 1] | Target: 0 | Prediction: 0
Inputs: [1, 0] | Target: 0 | Prediction: 0
Inputs: [1, 1] | Target: 1 | Prediction: 1
```



5. Learning rate = 0.01

```
Epoch: 1/500 | Average Error: 0.5 | Weights: [0.01, 0.01]
Epoch: 2/500 | Average Error: 0.75 | Weights: [0.02, 0.01]
Epoch: 3/500 | Average Error: 0.75 | Weights: [0.02, 0.01]
Epoch: 4/500 | Average Error: 0.5 | Weights: [0.02, 0.02]
Epoch: 5/500 | Average Error: 0.25 | Weights: [0.02, 0.01]
Epoch: 6/500 | Average Error: 0.0 | Weights: [0.02, 0.01]
Training completed.
```

```
Final weights: [0.02, 0.01]
Final bias: -0.03
Inputs: [0, 0] | Target: 0 | Prediction: 0
Inputs: [0, 1] | Target: 0 | Prediction: 0
Inputs: [1, 0] | Target: 0 | Prediction: 0
Inputs: [1, 1] | Target: 1 | Prediction: 1
```



B:

1. Learning rate = 1

Reached maximum epochs without meeting the error threshold.

Final weights: [4.69913252429422, 4.6916694687367775]

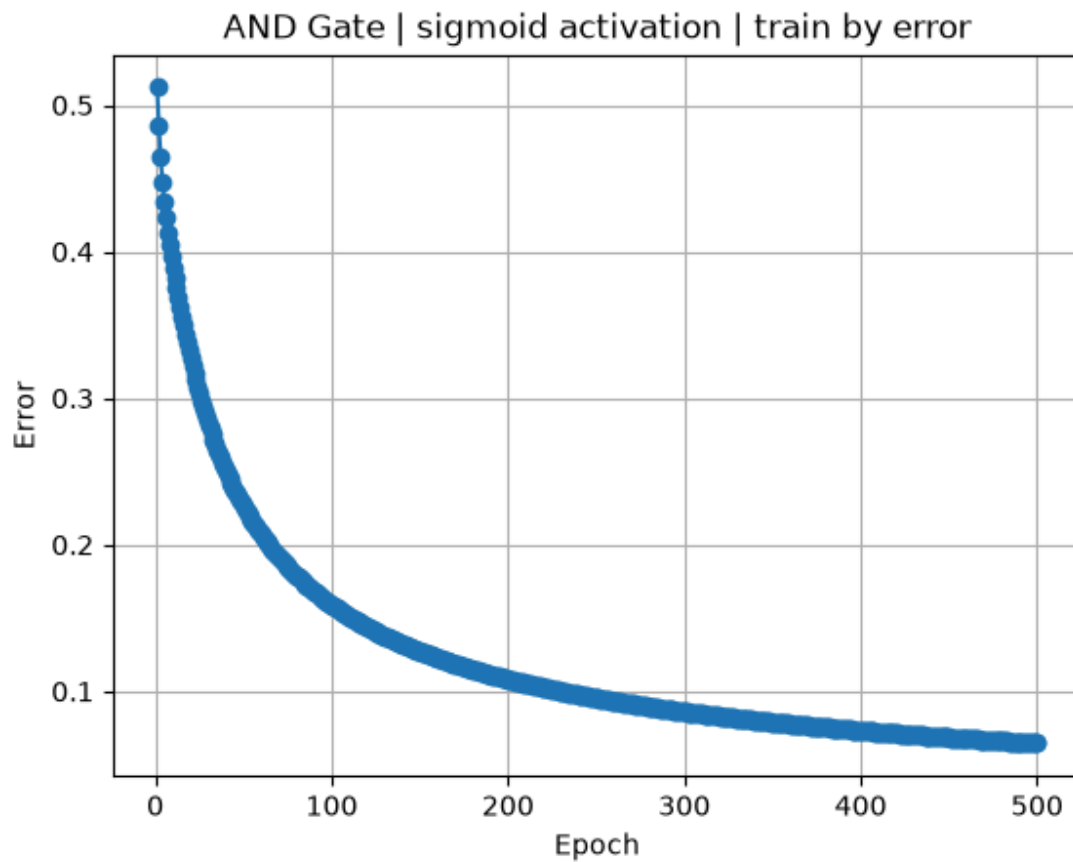
Final bias: -7.126439452748278

Inputs: [0, 0] | Target: 0 | Prediction: 0.0008029302532405585

Inputs: [0, 1] | Target: 0 | Prediction: 0.08055944867130635

Inputs: [1, 0] | Target: 0 | Prediction: 0.08111396763105083

Inputs: [1, 1] | Target: 1 | Prediction: 0.9058822387026457



2. Learning rate = 0.5

Reached maximum epochs without meeting the error threshold.

Final weights: [3.816832696615017, 3.809038559367979]

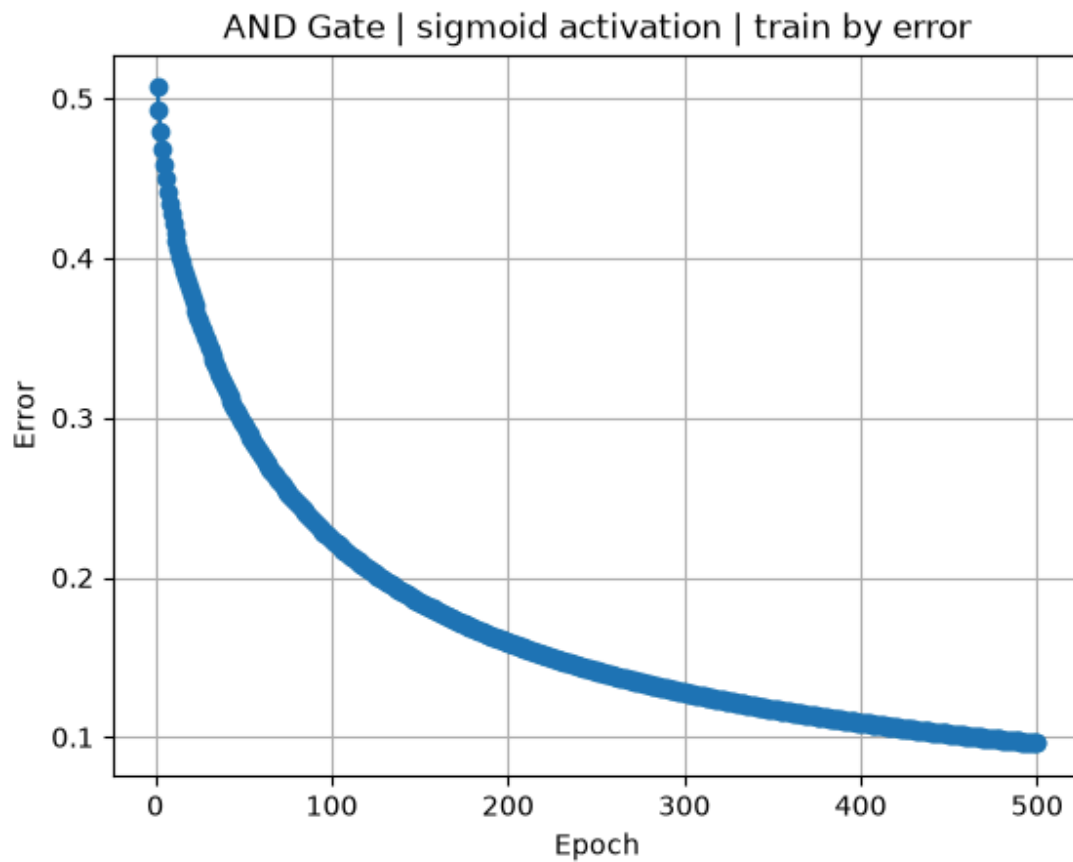
Final bias: -5.810030319698463

Inputs: [0, 0] | Target: 0 | Prediction: 0.002988381998042628

Inputs: [0, 1] | Target: 0 | Prediction: 0.11909883286794211

Inputs: [1, 0] | Target: 0 | Prediction: 0.11991898002584507

Inputs: [1, 1] | Target: 1 | Prediction: 0.8600663235885126



3. Learning rate = 0.1

Reached maximum epochs without meeting the error threshold.

Final weights: [1.8290110452583865, 1.8233387359417002]

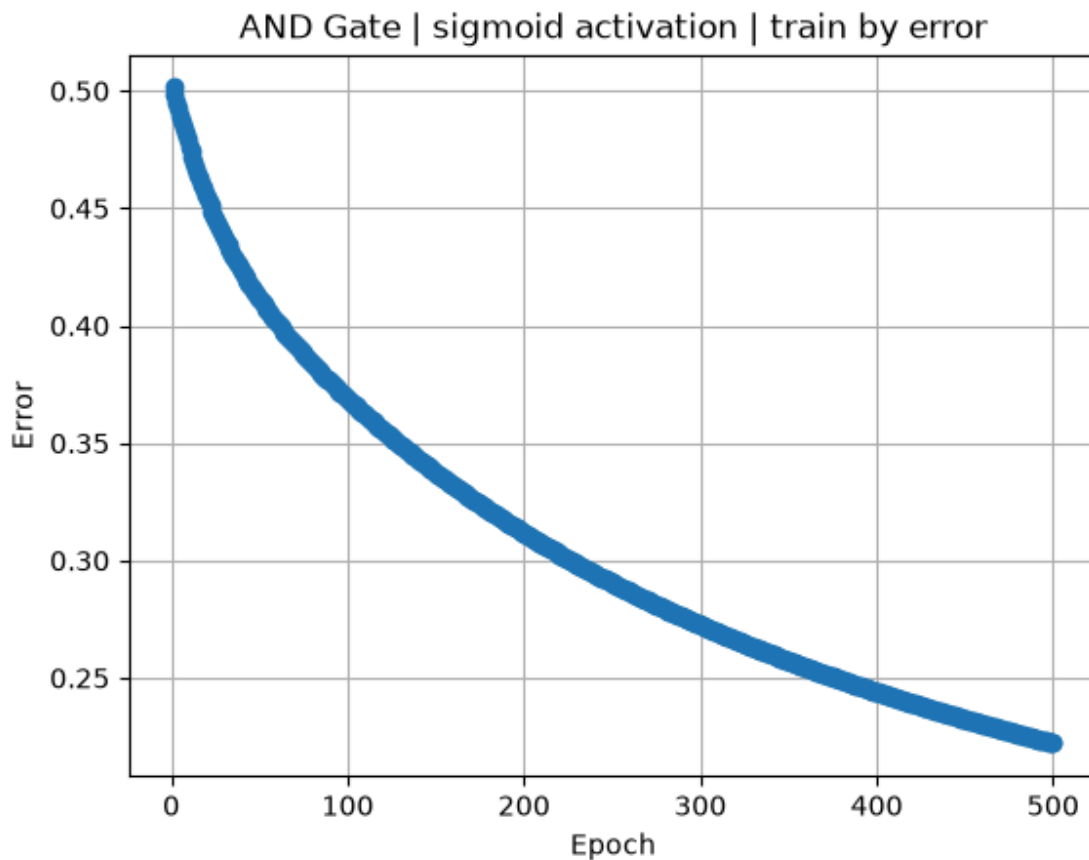
Final bias: -2.8859333724720053

Inputs: [0, 0] | Target: 0 | Prediction: 0.0528533234141035

Inputs: [0, 1] | Target: 0 | Prediction: 0.2568139285930563

Inputs: [1, 0] | Target: 0 | Prediction: 0.25789804112897935

Inputs: [1, 1] | Target: 1 | Prediction: 0.6827451816524243



4. Learning rate = 0.05

Reached maximum epochs without meeting the error threshold.

Final weights: [1.1593666290676214, 1.1562175324078188]

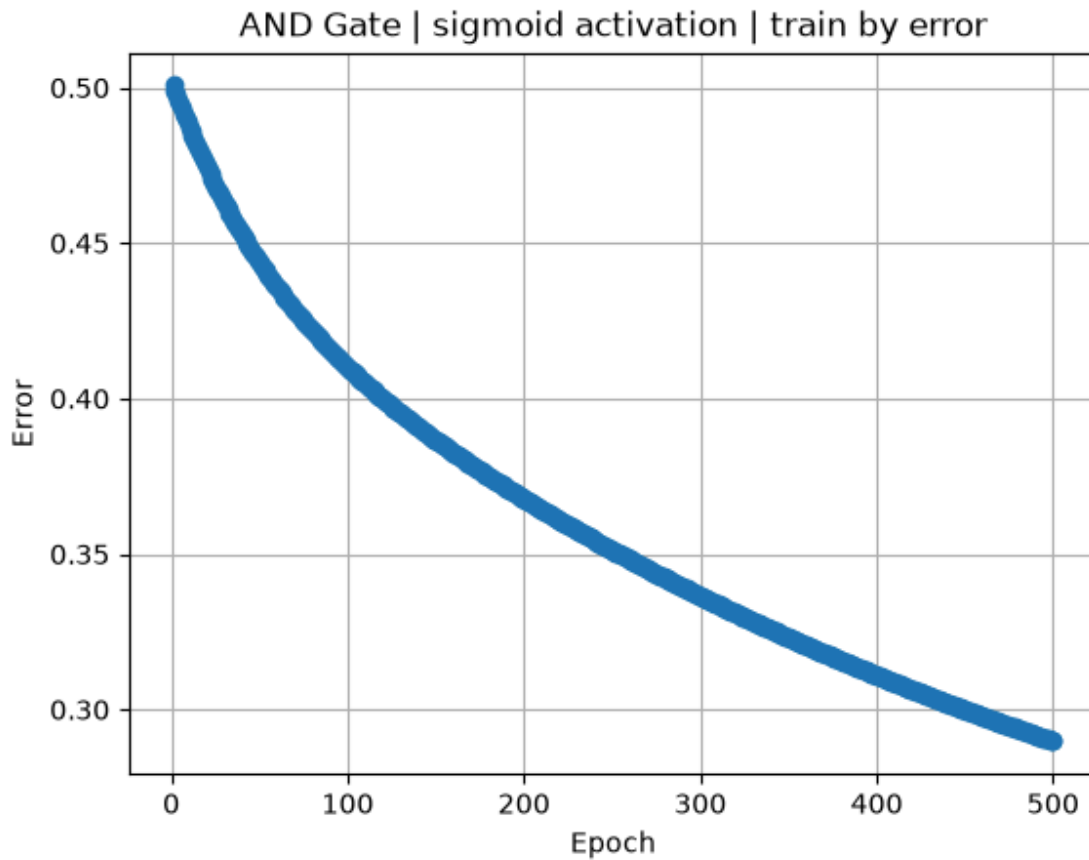
Final bias: -1.9547110847000224

Inputs: [0, 0] | Target: 0 | Prediction: 0.12404057031129295

Inputs: [0, 1] | Target: 0 | Prediction: 0.3103478548316665

Inputs: [1, 0] | Target: 0 | Prediction: 0.3110222647114342

Inputs: [1, 1] | Target: 1 | Prediction: 0.5892517648840917



5. Learning rate = 0.01

Reached maximum epochs without meeting the error threshold.

Final weights: [0.20232678540924984, 0.20202303879801364]

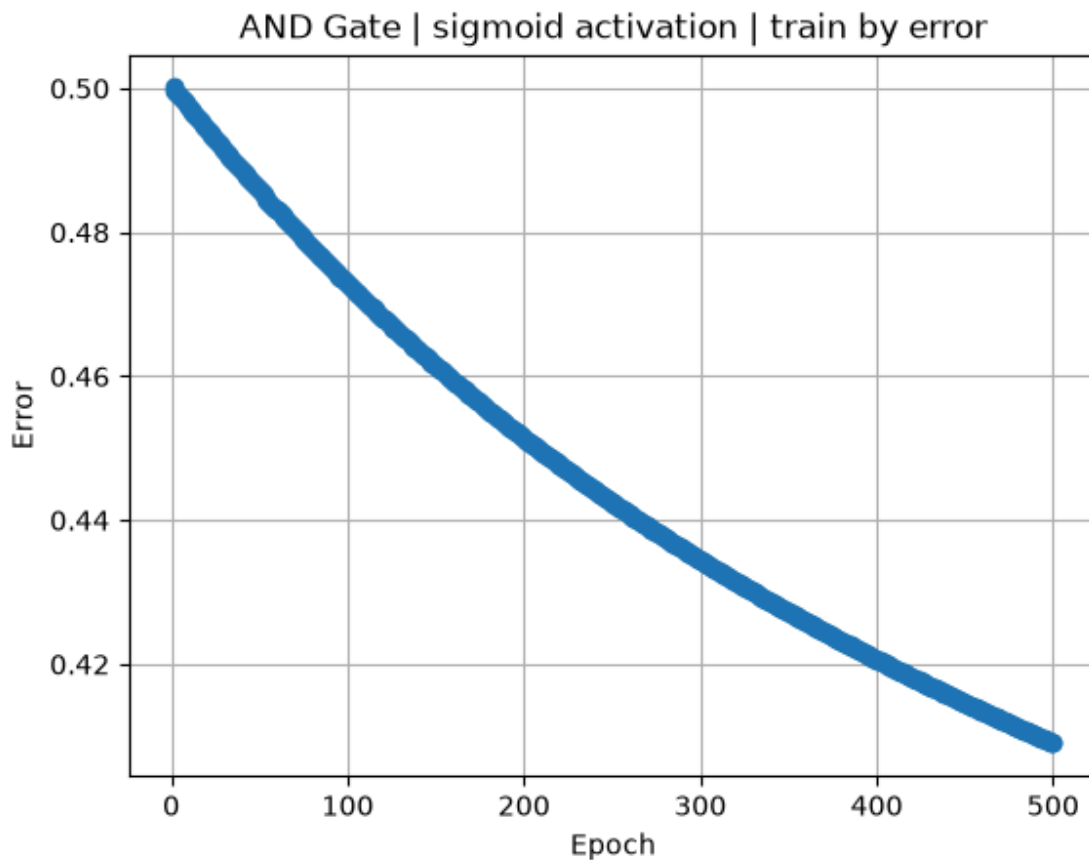
Final bias: -0.7588811476299432

Inputs: [0, 0] | Target: 0 | Prediction: 0.3188892304441765

Inputs: [0, 1] | Target: 0 | Prediction: 0.36427474391187475

Inputs: [1, 0] | Target: 0 | Prediction: 0.36434508804299554

Inputs: [1, 1] | Target: 1 | Prediction: 0.41228401995005853



- **Mechanism:**

1. AND (STEP) model:

I predicted that a smaller learning rate would need proportionally more epochs, but that's not what happened. All five learning rates (1, 0.5, 0.1, 0.05, 0.01) converged in about the same 4–6 epochs. The reason is in the update rule itself, $\Delta w = \eta(t-o)x$: since STEP's output and the target are both exactly 0 or 1, the error term $(t-o)$ is always exactly: 1, 0, or +1, so the learning rate η just scales every weight and bias update by the same constant factor (you can see this directly in the results at $\eta=1$ the final weights were [2, 1], and at $\eta=0.5$ they were exactly half that, [1, 0.5]). But STEP only checks whether the weighted sum is positive or negative, and multiplying everything by a positive constant never flips that sign, meaning that the model reaches the same

correct/incorrect decision on each training example regardless of how big or small η is, and ends up taking basically the same number of epochs either way. My prediction assumed the learning rate controls speed the same way it would for a smooth, continuous learning process, but for STEP it really only controls the final size of the weights, not how fast the decision boundary gets found (a direct result of the Perceptron Convergence Theorem, which bounds mistakes by the data's geometry, not the learning rate). The couple of epoch differences seen (4 vs. 6) are just floating-point rounding at the boundary, and the up-then-down error pattern within a run happens because online updates fixing one example can briefly misclassify another before the model settles.

2. AND (Sigmoid) model:

I predicted the same thing here like the STEP activation function. The smaller learning rate, more epochs, but still converging comfortably inside the 500-epoch limit. That prediction was wrong in a more serious way: none of the five learning rates reached the error threshold at all, and shrinking the learning rate didn't just slow things down proportionally, it made the model get stuck. The gradient descent update, $\Delta w = \eta(t-o) \cdot o(1-o)x$, includes the sigmoid's own derivative, $o(1-o)$, which is largest (0.25) when the output is near 0.5 and shrinks toward zero as the output gets closer to 0 or 1. Since sigmoid never actually reaches 0 or 1, hitting the error threshold requires pushing the weights to a fairly large size (at $\eta=1$, weights grew to about 4.7, and it still wasn't quite there), and that growth naturally slows itself down the closer it gets. The same update rule that moves the model forward also shrinks its own step size over time. A smaller learning rate multiplies on top of that self-slowness effect instead of just stretching it out evenly, so at $\eta=0.01$ the model was still stuck near its starting point after all 500 epochs, with outputs around 0.3–0.4 that barely distinguish the "0" cases from the "1" case. So the real mechanism

isn't "smaller rate, same result later", but was it's "smaller rate compounds with a gradient that's already shrinking, and the model can run out of epochs before it gets anywhere close."

- **Reconciliation:**

AND models (STEP)

I predicted: A smaller learning rate would require proportionally more epochs to reach the error threshold, since smaller weight updates should mean slower progress toward the correct decision boundary.

What actually happened: All five learning rates (1, 0.5, 0.1, 0.05, 0.01) converged in about the same 4–6 epochs, which showed that there was no meaningful slowdown as η shrank by two orders of magnitude, even though the final weights themselves scaled almost exactly with η (e.g., $\eta=1 \rightarrow w=[2, 1]$; $\eta=0.5 \rightarrow w=[1, 0.5]$).

What my mental model was missing: I was treating STEP like a smooth optimization process where step size directly controls how fast you approach the target. But STEP's decision only depends on the sign of the weighted sum, not its exact magnitude, and since the error term ($t-o$) is always exactly -1 , 0 , or $+1$, the learning rate just scales the whole weight trajectory by a constant factor, it never changes which side of the boundary an input lands on.

Updated rule of thumb: For the STEP/perceptron rule, the learning rate mainly controls the final magnitude of the learned weights, not the number of epochs needed to converge. Convergence speed is set by the geometry of the data (how separable it is), not by η , as long as η is positive and not so large it causes wild overshoot.

AND models (Sigmoid)

I predicted: The same thing was predicted here that a smaller learning rate would need proportionally more epochs but would still comfortably converge within the 500-epoch budget.

What actually happened: None of the five learning rates reached the 0.05 error threshold at all within 500 epochs, and the slowdown from a smaller η was far worse than proportional. At $\eta=1$ the model got reasonably close (outputs 0.906/0.081), but at $\eta=0.01$ it had barely moved from its starting point after the full budget (outputs still bunched around 0.32–0.41).

What my mental model was missing: I assumed the learning rate was the only thing controlling speed, treating the relationship as linear. I missed that sigmoid's own gradient term, $o(1-o)$, shrinks on its own as the output approaches the target. So the model is fighting a self-imposed slowdown on top of whatever η does. A small η doesn't just add delay, it multiplies with an already-shrinking gradient, which can leave the model stuck far from the answer rather than just arriving later.

Updated rule of thumb: For sigmoid/gradient descent, a smaller learning rate doesn't produce a predictable, proportional delay because the gradient itself vanishes as the output saturates toward 0 or 1, a low enough learning rate can cause the model to effectively stall and fail to converge within a fixed epoch budget.